SO2R Box — POC Bench Plan

Five milestones, ~4 evenings, to retire the architectural risk in the SO2R box design before committing to the full $325 BOM. Companion to `POC_PARTS_LIST.pdf`. Bench-only, breadboard-only, no soldering iron required until milestone 4.

How to use this plan

Each milestone names the wires you push into the breadboard, the test you actually run, the one signal that proves it worked, and the failure mode you are most likely to chase. **Milestone 4 (CAT control) is the riskiest leg of the entire build** — if a particular radio refuses to talk over USB-CDC, you want to know now, not after $325 of parts have arrived. Treat each milestone as a stop-gate: don't move forward until its success criterion is met.

Heltec V3 pin map used throughout

Pin assignments avoid the GPIO 33-38 PSRAM range called out in REVIEW B5. Confirm with a multimeter on your actual board before wiring — Heltec sometimes ships with PSRAM and sometimes without.

Heltec pin	Role	Used in milestone
`GPIO 41 (SDA)`	I2C SDA to MCP23017	2, 3, 5
`GPIO 42 (SCL)`	I2C SCL to MCP23017	2, 3, 5
`GPIO 17 (TX1)`	UART1 TX to CP2102N RX	4
`GPIO 18 (RX1)`	UART1 RX from CP2102N TX	4
`GPIO 4 - 7`	4x button inputs (INPUT_PULLUP)	1, 3
`GPIO 21`	WS2812B data line (3.3V drive)	3
`GPIO 8, 9, 10`	CD4051 address A, B, C	5
`Built-in OLED`	SSD1306 on I2C0 (separate from MCP)	1, 3, 4

Milestone 1 — Blink + OLED + button

Time budget: ~1 hour

Parts in play

Heltec V3, USB-C cable, breadboard, 1x tactile button, 1x WS2812B 5mm.

Goal

Confirm the toolchain: Arduino-IDE or PlatformIO can compile, flash, open the serial monitor, drive the built-in OLED, read a GPIO, and drive one NeoPixel from 3.3V. Establishes that the next four evenings are about *circuit* debugging, not toolchain debugging.

Wiring

From	To	Notes
`Heltec USB-C`	`Laptop`	Power + serial monitor.
`Button leg 1`	`Heltec GPIO 4`	Other leg to GND. Use INPUT_PULLUP.
`WS2812B pin 1 (DIN)`	`Heltec GPIO 21`	Drive directly from 3.3V — yes, it will work badly. That's the point.
`WS2812B pin 2 (+5V)`	`Heltec 5V (USB) pin`	USB bus voltage is fine for one LED.
`WS2812B pin 4 (GND)`	`Heltec GND`	

What you actually do

1. Install the Heltec ESP32 board package (Heltec docs link is in their GitHub).

2. Flash the stock `OLED_Hello` example. Verify the OLED prints text.

3. Add code: on button press, increment a counter on the OLED and cycle the WS2812B through red → green → blue → off.

4. Open the serial monitor at 115200. Print the button-press count to USB-CDC.

5. Touch the WS2812B with a finger while it's running — warm-finger trick: heat shifts the threshold, the LED will start dropping pixels or glitching. **This is REVIEW B6 in person.**

Success criterion

OLED shows a button-press counter that increments on every press. WS2812B responds — even if colours are wrong — on every press. Serial monitor prints to laptop. You have personally seen the WS2812B misbehave from a 3.3V drive.

Failure modes to watch for

• Heltec V2 vs V3 confusion: V2 is plain ESP32, V3 is ESP32-S3. Different board package, different pin map. The USB connector is the giveaway — V3 has USB-C.

• If the OLED is dark, the Heltec's `Vext` pin needs to be pulled LOW first to enable the OLED rail. Heltec's library handles this; bare ESP-IDF doesn't.

• If the serial monitor opens then closes immediately on flash, you've selected the USB-CDC port; close all serial windows during flash, reopen after.

Milestone 2 — I2C bring-up + relay click test

Time budget: ~1-2 hours

Parts in play

Heltec V3, MCP23017 breakout, 8-channel **5V** relay module, breadboard, jumper wires. **No 12V supply yet** — the 12V wall adapter is reserved for the final build with the real G2RL-1-E-CF-DC12 relays (12V coils). The POC relay module's coils are 5V, so the bench supply for this milestone is just the laptop's USB.

Goal

Validate the I2C control plane that runs the entire RF box. Click each of the eight relays on/off from the ESP32 with no glitches, no relay chatter on power-up, no I2C bus hangs. This is the architectural linchpin — if it works at 8 channels on a breadboard, it will work at 18 channels on a PCB.

Wiring

From	To	Notes
`Heltec GPIO 41 (SDA)`	`MCP23017 SDA + 4.7k to 3V3`	Pull-up is mandatory; some breakouts include it on-board.
`Heltec GPIO 42 (SCL)`	`MCP23017 SCL + 4.7k to 3V3`	Same as above.
`Heltec 3V3`	`MCP23017 VDD`	Logic supply for the GPIO expander.
`Heltec 5V (USB pin)`	`Relay module VCC + JD-VCC`	Option A: single 5V rail off USB powers logic + coils. Leave the JD-VCC jumper installed. 1 coil ~70 mA; running all 8 = ~560 mA, which is above the 500 mA USB-host limit — do the stress test under Option B.
`(optional) 5V USB charger -> USB-A breakout`	`Relay module JD-VCC (jumper PULLED)`	Option B: separate 5V coil rail. Mirrors the final RF box topology (logic and coil rails isolated so a coil collapse can't dip the MCU). Use this for the 60s stress test.
`Heltec GND`	`MCP23017 GND + relay module GND`	Common ground — mandatory even under Option B.
`MCP23017 A0/A1/A2`	`GND`	I2C address = 0x20.
`MCP23017 GPB0-7`	`Relay module IN1-IN8`	Drive the opto-isolator inputs directly from 3.3V. The opto's current-limit resistor handles the level mismatch.

What you actually do

1. Wire Option A first (everything off the Heltec's USB 5V rail). Power on with USB — relay module status LEDs light, coils should NOT energise yet.

2. Run an I2C scanner sketch. Expect to find 0x20 (MCP23017) and 0x3C (the on-board OLED).

3. Configure MCP23017 GPB as outputs (IODIRB = 0x00). Write 0x00 to OLATB. Watch for relay chatter — if any relay clicks on its own, the MCP outputs powered up high-impedance and the inputs floated. This is the issue REVIEW flags about the ULN2803A power-up state, manifesting here on the relay-module clones. Add 10k pull-downs to GND on IN1-IN8.

4. Polarity check. Most 8-ch relay modules are active-low (IN pulled to GND energises the relay). Write 0x01 to OLATB — if relay 1 clicks, the module is active-high; if relay 1 stays put but the OTHER seven click, it's active-low. Note the polarity and invert the bitmask in firmware accordingly.

5. Walk a single bit through GPB0..GPB7 with a 200 ms delay (binary 0x01, 0x02, 0x04, ... 0x80, with the polarity correction from the previous step). Each relay clicks in sequence.

6. Switch to Option B for the stress test: plug the relay module's JD-VCC into a 5V USB charger via a USB-A breakout, pull the JD-VCC jumper. Run a tight loop toggling all 8 outputs as fast as the I2C bus allows for 60 seconds. Verify no missed clicks, no MCU brown-out (USB rail no longer collapses under coil inrush).

7. (If you bought the logic analyzer) Capture SDA/SCL during the walking-bit sequence and confirm clean rise edges, no NACKs.

Success criterion

All 8 relays click on command, in any order, with no chatter on power-up or power-down, sustained for 60+ seconds of stress testing under Option B (separate coil supply). I2C scan shows exactly the expected devices. Polarity is documented in firmware.

Failure modes to watch for

• **Do not feed 12V into a 5V relay module's JD-VCC.** The 5V coils will overheat and stick closed within minutes. The 12V adapter from POC_PARTS_LIST belongs in the final-build pile, not on this bench.

• Counterfeit MCP23017s exist on Amazon. Symptom: I2C scan finds 0x20 but writes to GPIO have no effect. Swap to an Adafruit-branded listing.

• Relay-module JD-VCC jumper: Option A leaves it installed (one rail), Option B pulls it (two rails). Pick one explicitly; halfway between the two creates a short.

• If 3.3V GPIO won't reliably drive the opto inputs (a few clones use a too-large current-limit resistor sized for 5V logic), the workaround is to keep VCC at 5V from USB — the opto LED then has the full 5V across the resistor when the IN pin is pulled low by the MCP, and reliably illuminates regardless of the 3.3V high level on the IN net.

• Cheap modules sometimes ship with the wrong relay populated. Read the relay can — you want SRD-05VDC-SL-C (or similar 5V-coil part). If you see SRD-12VDC on the can, the module was mis-shipped; do NOT power it from USB.

Milestone 3 — Panel UI + state machine

Time budget: ~1 evening

Parts in play

Everything from milestone 2, plus 4x tactile buttons, 4x WS2812B 5mm.

Goal

Wire up a working operator panel: four buttons (Radio Select x2, Band-, Band+), four WS2812B status LEDs, plus the Heltec OLED showing current band + active radio. Pressing buttons commands relay clicks. This is the milestone where the SO2R box stops being a parts pile and starts feeling like the actual product.

Wiring

From	To	Notes
`4x button leg 1`	`Heltec GPIO 4, 5, 6, 7`	INPUT_PULLUP. Other leg of each button to GND. Add 10 nF GPIO-to-GND for RF immunity per REVIEW minor item.
`WS2812B chain DIN`	`Heltec GPIO 21`	First LED's DIN; subsequent LEDs daisy-chain DOUT -> DIN.
`WS2812B +5V`	`Heltec 5V`	100 uF bulk cap across the LED supply, close to the first LED.
`MCP23017 -> relays`	`(as milestone 2)`	

What you actually do

1. Implement a state machine: state = (active_radio: 1|2, band: 160m..10m). Buttons mutate state.

2. On state change, write a new bit pattern to MCP23017 OLATB — mapping bands 160m..10m to GPB0..GPB7.

3. Drive the four WS2812B LEDs to reflect: active radio (1 LED), PTT state (1 LED), band group (low/high/WARC; 2 LEDs).

4. Update the OLED on every state change: line 1 = 'RIG 1' / 'RIG 2', line 2 = current band, line 3 = 'PTT' indicator.

5. (REVIEW B6 again) Pull the WS2812B +5V down to 3.3V on the chain — watch colours go wrong. Now you have visceral proof you need the 74AHCT125 level shifter in the final build.

6. Debounce: confirm rapid button mashing produces exactly the intended state transitions, not skips or doubles. ~20-50 ms software debounce is usually enough.

Success criterion

Pressing any button reliably triggers (a) a relay click corresponding to the new band, (b) the correct OLED state, (c) the correct WS2812B colour. Each happens within 50 ms of the press. No button mash produces wrong state.

Failure modes to watch for

• WS2812B chain timing is RMT-driven on ESP32-S3. The Arduino library 'FastLED' uses RMT under the hood; if you see corrupted colours *only* after a serial print or WiFi join, you've collided with RMT — pin the LED task to core 0 or switch to the I2S-driven driver.

• Buttons against rotating-field interference (i.e. when you eventually use this near a transmitter): a 10 nF GPIO-to-GND on each input is the difference between a clean POC and a panel that fires phantom presses.

Milestone 4 — original scope (DEPRECATED)

Was: “ESP32 talks CAT directly to the radio via an external CP2102N USB-C UART module.” Phases A (UART loopback) and B (PC ↔ radio direct with Hamlib) passed cleanly. **Phase C failed for fundamental topology reasons** against the Yaesu FTX-1 Optima (and any modern transceiver):

• **The radio's USB-CAT port is a USB device port**, not a host. Two USB devices cannot communicate without an intermediate host. A CP2102N plugged into the radio's USB jack does nothing - the radio won't enumerate it. The original wiring diagram was physically impossible to use.

• **The radio's hardware-UART CAT port (CAT-3 on a mini-DIN-10 TUNER/LINEAR jack) is 5V CMOS**, above the ESP32-S3's 3.3V V_{IH} headroom on TX and above the chip's 3.3V absolute-max on RX. Direct wiring needs a bidirectional level shifter (TXS0108E or BSS138 board) plus a custom mini-DIN-10 cable - not cheaper than abandoning the path.

• **A CP2102N cannot bridge UART ↔ UART.** It converts USB ↔ UART. To bridge between an ESP32 UART and a radio UART you need a host in the middle (a PC) - which is exactly what the new M4 architecture embraces.

Milestone 4 — PC ↔ box USB-CDC protocol (NEW SCOPE)

Time budget: ~1 evening

Parts in play

Heltec V3 (powered via its onboard USB-C / CP2102N - **no external CP2102N needed**), the PC running CW Sweep or the Python bench client (`scripts/so2r_client.py`), a real HF radio if you want to close the integration loop, otherwise the MCP23017 + relay module from M2 is enough.

Goal

Prove the PC-driven control plane. All CAT lives on the PC (Hamlib / rigctld / Sweep's RadioManager); the box is a smart USB-CDC peripheral that owns the antenna relay matrix, PTT lines, audio routing, and panel UI. The PC tells the box which band each radio is on; the box reports panel button events back to the PC. This decouples the box from per-vendor CAT protocols and matches the topology constraints of real radios.

Wiring

From	To	Notes
`Heltec USB-C`	`Mac USB port`	Onboard CP2102N is the PC link. No external UART hardware.
`(MCP23017 + relay module)`	`(as M2 wiring)`	Unchanged from milestone 2.
`Radio CAT cables`	`Mac (separately)`	Each radio talks CAT to the PC directly via its own cable/library. Not connected to the box at all.

What you actually do

1. Phase A — protocol smoke test on the bench. With the box flashed and the MCP23017 + relay module wired (M2 state), run `uv run scripts/so2r_client.py` on the Mac. Type `ping` -> expect `PONG`. Type `state` -> see active radio, both bands, both PTT flags, relay mask. This proves the USB-CDC link + parser on both sides.

2. Phase B — relay drive via protocol. From the client, run `band 1 20m`, `band 1 40m`, `band 1 15m` in sequence; verify the matching relay clicks on the M2 hardware. Then `band 2 80m` and verify the radio-2 antenna relay clicks instead (when the active radio switches). Proves the per-radio state model is correctly wired through to the MCP23017 outputs.

3. Phase C — panel events. Press BTN1-BTN4 on the panel. Each press should arrive at the client as an unsolicited `EVT BTN n PRESS` line, and any operator-driven state change (e.g. cycling the band) should arrive as `EVT STATE BAND1=20m`. Confirm the client's state model stays synchronised after panel-driven changes.

4. Phase D — PC-driven frequency follow. Tell the client a frequency (`freq 1 14250`) and verify it derives the band correctly and pushes the right `BAND` command. This is the building block for Sweep's eventual integration: any radio frequency change Sweep observes via Hamlib → `set_band_from_frequency()` → the box's relay matrix follows.

5. Phase E — PTT round trip. `ptt 1 on` / `ptt 1 off`; verify the box accepts and reports state. In v1 the PTT state is logical only (no hardware drive yet); when PTT-out pins are wired in a later milestone, the firmware just adds GPIO drive at the right point and the protocol is unchanged.

Success criterion

PC client can query and mutate the box's state over USB-CDC, the MCP23017 relay outputs follow band commands within ~10 ms, panel button presses arrive at the PC as EVT lines, and the client's state mirror stays consistent with the box after any combination of PC-driven and operator-driven changes. **No external CP2102N module, no per-radio CAT cabling to the box, no level shifters.**

Failure modes to watch for

• The 8-band slot model on the box (160/80/40/20/15/10/WARC/BYP) is narrower than Hamlib's full band list. The Python client's `band_for_frequency()` maps 30m/17m/12m/60m to WARC and 6m/2m/70cm to BYP. If you operate WARC bands competitively, the box's antenna matrix will need a wiring revision later.

• Asynchronous EVT lines and synchronous OK / ERR / PONG replies share the same USB-CDC stream. The Python client uses a reader thread to split them; if you re-implement on another stack (Swift / Rust) the same dispatch discipline applies - don't try to do blocking reads from the main loop.

• When the box boots, it emits `EVT BOOT version=N`. The client should treat that as a hint to re-query `STATE` on connect so its mirror is correct even if the box reset out from under it.

• The Heltec's onboard CP2102N enumerates as `/dev/cu.usbserial-XXXX` on macOS, NOT `usbmodem`. `scripts/so2r_client.py` auto-detects this prefix; if you symlink or rename the device the auto-detect will miss it.

Milestone 5 — Audio mux click-through (STRETCH)

Time budget: ~1 evening

Parts in play

Heltec V3, 1x CD4051BE (DIP-16), 2x 3.5mm audio jack breakouts, smartphone, headphones.

Goal

Prove the CD4051 analog mux concept: a single 8:1 mux fed from one source, with three GPIO lines selecting one of eight inputs (only one used for POC), output piped to a headphone. Click-free switching at audio rates.

Wiring

From	To	Notes
`Phone audio out`	`CD4051 X0 (pin 13)`	One channel only; ground both phone L/R and the breadboard ground.
`Heltec GPIO 8, 9, 10`	`CD4051 A, B, C (pins 11, 10, 9)`	Address inputs.
`CD4051 INH (pin 6)`	`GND`	Enable the mux.
`CD4051 VDD (pin 16)`	`Heltec 3V3`	Single-supply operation; audio rides on a virtual ground.
`CD4051 VEE/VSS (pins 7, 8)`	`GND`	
`CD4051 X (pin 3)`	`Headphone tip`	Series 10 uF cap, then 10 k to GND on the headphone side — this is REVIEW's audio coupling reminder in miniature.

What you actually do

1. Play a continuous tone from the phone (any frequency-generator app).

2. Set CD4051 address to 0 (A=B=C=0) — X0 connects to the X common pin. Tone should be audible in the headphone.

3. Switch the address to 1, 2, 3 — tone should go silent (no input wired to those channels).

4. Switch back to 0 — tone returns. Listen for a click or pop on the transition; CD4051 'break-before-make' should give a clean silence on the inactive channels.

5. Optional: wire a second source (a function generator or another phone) to X1, alternate addresses, listen for the mux switching between the two sources without obvious clicks.

Success criterion

Tone is audible only when the address selects the input that has signal on it. Switching between channels produces no audible click. Audio quality is acceptable for monitoring (POC quality; final build has the MCP6002/NJM4556 headphone driver).

Failure modes to watch for

• CD4051 needs the audio to ride *between* its VDD and VSS rails. Phone audio is typically AC-coupled around 0V — the negative half-cycle will be clipped. For POC this is fine (you'll still hear the tone, just distorted on negative peaks); for the final build the design uses a buffered virtual ground at VDD/2 (REVIEW's TLE2426 recommendation).

• If you hear strong hum, you forgot to common the phone's ground to the breadboard ground.

• If you hear nothing on any address, your phone's headphone-out auto-detected the load as too low. Add a 33 k from input to ground to raise the impedance.

After all milestones pass

You now know, with bench evidence, that:

• The I2C control plane works at 8 channels and will scale to 18.

• The Heltec V3 GPIO assignments you picked don't collide with PSRAM on your specific board (REVIEW B5).

• WS2812B at 3.3V is unreliable — the 74AHCT125 in the final BOM is genuinely required (REVIEW B6).

• Your specific radio talks CAT cleanly over USB-CDC through the CP2102N.

• The CD4051 mux concept is sound.

Now is the right time to order the full $325 BOM.

If any milestone fails

Stop. Don't order the full BOM until you understand why. A failure here costs $5 and an evening; a failure after $325 of parts arrive costs days and may require a re-spin of the PCB design.

Bench-safety reminders

• Use a dummy load every time you key PTT during milestone 4. Never into an antenna.

• The 12V wall-wart adapter has no isolation — respect it as if it were mains.

• Anti-static wrist strap when handling the CD4051 and MCP23017 directly — both are CMOS and the breakout boards don't protect the IC pins you wire to.